

AI Context Engineering: A Working Guide...for now

The core thesis

Two ideas drive everything else:

1. **Code is better context than human-written documentation.** The single best source of truth about how a library behaves is its source code, not its docs. Docs drift, omit edge cases, and are written for humans skimming — source is exact.
2. **Less context beats more context.** A model degrades as its context window fills. The goal is to give it *just enough* — the exact function, the exact constraint — not the whole codebase.

These are a reversal of the prevailing 2024–2025 approach, which was to stuff the agent with everything (minifying entire codebases into XML, vector-indexing the whole repo with RAG). Modern models search well enough that you point them at the right place rather than pre-loading everything. This reversal is real and worth internalizing: the techniques in this space have roughly a six-month half-life.

The workflow

The whole thing is a loop that ends in a verifiable state. Each piece exists to keep context small and the end condition clear.

1. Plan first — for your accountability, not the agent's.

Have the model generate a plan before it writes code. The value is that *you* can catch a too-large scope before any code exists. A model writing a plan does not account for its own context window; it will dump everything needed for the whole task into one plan. When the plan looks big, push back: "break this into the smallest reviewable PRs you can." Small chunks keep each generation step inside the context sweet spot.

2. Give it exact context, not documentation.

Instead of pasting docs or letting the agent guess at an API, fetch the actual source of the dependency into the repo and tell the agent to reference it. (See `opensrc` below.) The agent finds the exact function — no guessing.

3. Generate the feature in a small PR.

Small, single-purpose PR. Resist the urge to bundle.

4. Run an external review loop until it passes.

A separate review agent scores the PR. Re-run the fix loop until the score clears your bar. The loop self-terminates on a defined end state (a passing score, a passing test) rather than running forever. This is the Karpathy "auto-research loop" idea applied to shipping code.

5. Restructure after the feature lands.

Agents tend to rewrite existing functions rather than reuse them, which accumulates duplicate code ("code smell") and makes future debugging harder. After a feature, run a pass that consolidates repeated logic into a reusable service layer. Cleaner structure is easier for the *next* agent session to pick up.

The principle tying it together: **you stay in charge**. The model predicts the next token; it has no model of whether its decision is correct. Ask any codebase "what are the issues here" and it will happily invent ten. Your judgment is the part that doesn't come in the box.

The verified tool stack

opensrc — exact source as context

`npx opensrc <package>` clones a dependency's source into `./opensrc/` and adds a short prompt block telling the agent the source is available for "understanding how a package works internally, not just its types/interface."

- Real tool, by **vercel-labs**. (The podcast's auto-transcript renders it as "open source," which is a homophone, not the license term.)
- Works beyond npm: `npx opensrc pypi:requests`, `npx opensrc crates:serde`, `npx opensrc owner/repo`.
Relevant to Python / Rust / Lisp-adjacent work, not just JavaScript.
- Usage pattern: fetch the source, then in your prompt tag the folder and say "reference the codebase" so the agent reads real implementation instead of guessing.
- Repo: <https://github.com/vercel-labs/opensrc>

Greptile — AI PR review with a confidence score

Indexes the whole repo into a semantic map, then reviews each PR with multi-hop investigation (tracing dependencies, checking git history across files). Every review gets a **0–5 confidence score** based on issue severity, change complexity, and how well the change fits existing patterns.

- Real tool. v3+ runs on the Claude Agent SDK.
- The "loop until 5/5" workflow is legitimate: a fix-and-re-review skill keeps iterating until the score clears.
- **Caveat the podcast omits:** independent benchmarks show Greptile has both the highest catch rate *and* the highest false-positive rate among AI reviewers. Greptile's own docs say the score is a **triage signal, not a rubber stamp** — a high score means it found nothing concerning, not that nothing is

wrong. Looping blindly to 5/5 can mean chasing phantom findings. Use the score to decide where to spend attention.

- Pricing as of early 2026: ~\$30/user/month. Self-hostable in-VPC.

Cursor + frontier models

The harness (Cursor) wraps the model with tools, a system prompt, and config files. The podcast's stack: Cursor as harness, switching between **GPT-5.5** (backend/architecture, "extra high" reasoning) and **Opus 4.7** (UI/frontend, "max" reasoning).

- All real and current. **Opus 4.7** shipped April 16, 2026 as Anthropic's flagship. **GPT-5.5** is real; "extra high" / `xhigh` is a genuine reasoning-effort setting. "Opus 4.7 Max" = the model plus Cursor's max reasoning setting, not a separate model.
- **Unverified claim:** the podcast asserts Cursor's harness benchmarks ahead of Claude Code and Codex on the same models. No source given; treat as a preference, not an established fact. Harness experience genuinely varies, but "Cursor wins" is the speaker's opinion.

Convex — backend as code

Pitch: everything (schema, scheduled functions, APIs) is TypeScript code, so the agent has full context on the backend without screenshots of a dashboard. The dashboard is only for production setup and billing.

- The "everything is code = full agent context" framing is the real selling point and is accurate.
- Note the tension with a Supabase workflow: Supabase mixes code with dashboard-managed config (RLS policies, etc.), which means some backend state lives where the agent can't directly see it. That's the gap Convex is arguing against. Whether it's worth switching is a separate question — the *context* argument is the relevant one here.

The one security tip worth adopting now

Tell your agent: never install a package younger than ~14 days.

Most supply-chain attacks are malicious package versions that drop and get caught within days. A 14-day floor means the vulnerability is usually flagged before you'd pull it. Directly relevant to anyone running automated pipelines (n8n, Supabase, GitHub Actions) where dependencies update without a human in the loop.

Adjacent verified-sensible advice from the same section: 2FA via an authenticator app, not SMS (SIM-swap is real); a password manager; and when a breach hits your feed, paste the details to your coding agent and have it check whether the affected package is in your project.

What to discount

- **The funnel.** "Click the link, it's free, I'll email you everything" repeated ~6 times means the product is your email address. The content is the bait.
- **"Be delusional."** Survivorship bias. You hear about the delusional founders who shipped; you don't hear about the larger number who shipped broken things and vanished. "Launch early, get feedback, iterate" is sound. "Believe your half-working MVP will change the world" is selection effect dressed as strategy.
- **Unsourced benchmark claims.** Any specific "X beats Y" number stated without a citation — treat as vibes until verified.

Quick reference

Thing	What it is	Verified?
opensrc	<code>npx</code> tool, fetches dep source as context (npm/pypi/crates/github)	✓ vercel-labs
Greptile	AI PR reviewer, 0–5 confidence score, loop-to-pass	✓ (watch false positives)
Cursor	Harness; switch models per task	✓
Opus 4.7	Anthropic flagship, strong on UI/frontend	✓ Apr 16 2026
GPT-5.5 (xhigh)	OpenAI model, strong on backend/architecture	✓
Convex	Backend-as-code, full agent context	✓ (positioning)
14-day package floor	Supply-chain defense heuristic	✓ sensible